2027 IEEE 4th International Student Conference on Digital Generation (DG)
24–26 February, 2027, Astana, Kazakhstan

Confidence-Aware Graph Neural Networks for Social Bot Detection: Message-Passing Reweighting and Similarity-Graph Construction

Kanalbay Kassymkhan
School of Software Engineering
Astana IT University
Astana, Kazakhstan
255479@astanait.edu.kz

La Lira L'vovna
L.N. Gumilyov Eurasian National University
Astana, Kazakhstan
Lira.La@astanait.edu.kz

***Abstract*—Graph neural networks are increasingly used to detect social bots, but a standard graph convolutional network weights neighbor messages by node degree only. We study confidence-aware message passing, in which every neighbor is reweighted by the model's own predictive confidence from the previous epoch, and three extensions: an adaptive variant with a learnable exponent applied on top of the baseline normalization, agreement-based edge weights, and pseudo-label self-training. Using a labeled Twitter dataset of 5000 accounts, repeated cross-validation, paired significance tests and ablations, we compare these models with feature-only, GCN, GraphSAGE and GAT baselines and study how the similarity graph between accounts is built. Graph construction is the decisive design choice. On a nearest-neighbor graph of profile descriptions every convolutional model is worse than a feature-only network, whereas on a graph of account metadata every graph model outperforms it. GraphSAGE on the metadata graph reaches an F1 score of 0.840 and an area under the ROC curve of 0.921, which is 3.2 F1 points above the feature-only model and 9.9 points above the original pipeline. With only 250 to 500 labeled accounts, GraphSAGE with confidence-based self-training is the best model. Confidence reweighting itself is correct and inexpensive but does not outperform the baselines, and its learned exponents shrink below one.**

***Index Terms*—social bot detection, graph neural networks, message passing, confidence weighting, self-training, graph construction**

I. Introduction

Social bots inflate follower counts, manufacture trends and spread disinformation at scale [1]. Because bots operate in coordinated clusters and reuse templated text, relational structure carries signal that a per-account classifier cannot see, which has motivated graph neural networks (GNNs) for bot detection [2]–[4]. A graph convolutional network (GCN) [5], however, weights each neighbor by degree only: an ambiguous account near the decision boundary contributes as much as one classified with near-certainty, so noisy signal can leak across the class boundary.

Our earlier work [6] proposed a Confidence-Aware GCN (Conf-GCN) that scales each neighbor's message by the model's own softmax confidence and reported, on a single split, a 3.0-point precision gain at a 7.6-point recall cost. This paper tests that claim rigorously and extends the method. We ask: (RQ1) is the gain real, and is it caused by the confidence signal? (RQ2) do adaptive extensions, namely a learnable exponent on top of the baseline normalization, agreement-based edge weights and self-training, help? (RQ3) how much does the construction of the similarity graph matter? (RQ4) what changes when only a few hundred accounts are labeled?

Our contributions are: (1) an adaptive family of confidence-aware layers (AC-GCN, AC-SAGE) that reduces exactly to GCN or GraphSAGE when the exponent is zero; (2) an evaluation with 25 runs per model, paired Wilcoxon tests with Holm correction and ablations that separate confidence from its confounders; (3) a graph-construction study showing that a metadata graph turns GNNs from worse than a feature-only model into the best detectors, improving F1 by 9.9 points over the original pipeline; (4) a label-scarce study in which confidence-based self-training gives the best detector; and (5) public code and per-run results[1].

II. Related Work

Feature-based detectors such as Botometer combine many profile and activity features in supervised ensembles [7]; they degrade as bots imitate humans and generalize poorly across datasets [8], [9]. Graph-based detectors propagate information along social or similarity edges with GCN, GraphSAGE [10] or relational models [3], [4], evaluated on benchmarks such as TwiBot-20 [11]. Graph attention networks (GAT) learn edge weights from features [12], and the benefit of message passing depends strongly on homophily [13]. ConfGCN [14] learns per-node label-distribution confidences with extra parameters; we instead use the classifier's own softmax output with no extra parameters. Confident predictions are also used as pseudo-labels in self-training [15], [16], while softmax

[1]https://github.com/solodancer235-star/confidence-aware-gcn-bot-detection

confidence is known to be imperfectly calibrated [17]. For tabular data without explicit links, a k-nearest-neighbor (k-NN) graph over embeddings [18] is a common substitute [19].

III. Method

A. Confidence-Aware GCN

Let $c_j = \max_k \mathrm{softmax}(z_j)_k$ be the confidence of node j computed from its logits z_j in the previous epoch ($c_j \in [0.5, 1]$ for two classes). Conf-GCN replaces the degree normalization of GCN by

$$h_i^{(l+1)} = \sigma\Big(\sum_{j \in \mathcal{N}(i)} \frac{c_j}{\sum_{k \in \mathcal{N}(i)} c_k} W^{(l)} h_j^{(l)} \Big). \qquad (1)$$

Confidence is initialized to one, recomputed after every epoch by a no-gradient evaluation pass, and never receives gradients; it uses no labels. The layer adds no parameters, and for equal confidences it reduces to mean aggregation without a self-loop, which is *not* the symmetric normalization of GCN. Since $c_j \in [0.5, 1]$, two neighbors of a node differ in weight by at most a factor of two.

B. Adaptive Confidence-Aware GNNs

To isolate the confidence signal, AC-GCN keeps the GCN weights $a_{ji} = 1/\sqrt{\hat{d}_i \hat{d}_j}$ over $\mathcal{N}(i) \cup \{i\}$ and only redistributes their mass:

$$w_{ji}^{(l)} = a_{ji}\, c_j^{\alpha_l} \frac{\sum_k a_{ki}}{\sum_k a_{ki}\, c_k^{\alpha_l}}, \qquad \alpha_l = \mathrm{softplus}(\rho_l) \geq 0. \qquad (2)$$

With $\alpha_l = 0$ or constant confidence the layer equals GCN exactly; the three learnable exponents (one per layer, initialized at one) let the model amplify or ignore confidence and remove the factor-two bound. AC-SAGE applies the same idea to the neighbor mean of GraphSAGE, $h_i' = W_r h_i + W_n \sum_j \tilde{w}_{ji} h_j$. The *agreement* variant replaces $c_j^{\alpha_l}$ by $(\sum_k p_{jk} p_{ik})^{\alpha_l}$, the probability that both endpoints share a class, which can switch edges off entirely. In *self-training* (+ST), after 20 warm-up epochs every unlabeled node with confidence of at least 0.9 receives its predicted class as a pseudo-label with loss weight 0.5.

C. Similarity-Graph Construction

The dataset has no follow graph, so we compare three label-free symmetrized cosine k-NN graphs ($k = 10$): over description embeddings (*description graph*, edge homophily 0.617), over the ten z-scored metadata fields (*metadata graph*, 0.777), and over both (*feature graph*, 0.756).

IV. Experimental Setup

Data. We use the Twitter human/bot dataset [20] (labels from Botometer and manual verification) in a cleaned English-only redistribution [21] with 29 407 accounts, and draw a balanced stratified subsample of 5000 accounts. Node features (394 dimensions) are a 384-dimensional Sentence-BERT embedding of the description [18], six log-transformed numeric fields standardized on training nodes, and four binary flags.

Models. A feature-only multilayer perceptron (MLP), GCN, GraphSAGE (mean), GAT and our models, all with three layers, hidden size 64 and dropout 0.5, implemented in PyTorch Geometric [22]. Learning rate and weight decay (and GAT heads) are selected by validation F1; extensions reuse the setting of their base model; training runs for 100 epochs.

Protocol. Stratified 5-fold cross-validation repeated over 5 seeds (25 runs per model, 60/20/20 train/validation/test). We report precision, recall and F1 for the bot class and the area under the ROC curve (AUROC), and compare models with paired Wilcoxon signed-rank tests with Holm correction [23], [24]. The label-scarce study trains on 100, 250 or 500 labeled accounts over 10 random splits.

V. Results

A. Conf-GCN Under Repeated Cross-Validation (RQ1)

Table I shows that the single-split precision gain does not replicate: Conf-GCN and GCN have the same precision (0.732 vs. 0.731, $p = 0.83$), Conf-GCN loses 0.9 F1 points ($p = 0.004$) and 2.5 AUROC points, and the feature-only MLP is the best model. The run-to-run standard deviation of F1 (about 0.02) is as large as the earlier single-split differences.

TABLE I
Description graph, 25 runs (mean ± std). p: paired Wilcoxon of F1 vs. Conf-GCN, Holm-corrected.

Model	Prec.	Rec.	F1	AUROC	p
MLP (no graph)	0.805	0.809	0.807±0.010	0.890	<0.001
GCN	0.731	0.752	0.740±0.016	0.819	0.004
GraphSAGE	0.805	0.801	0.803±0.014	0.887	<0.001
GAT	0.744	0.726	0.734±0.020	0.819	0.230
Conf-GCN	0.732	0.732	0.731±0.019	0.794	—

TABLE II
Ablations of Conf-GCN, description graph (25 runs). p: F1 vs. full model, Holm-corrected.

Variant	F1	AUROC	p
Conf-GCN (full, $\alpha = 1$)	0.731	0.794	—
Uniform confidence ($c \equiv 1$)	0.736	0.795	0.026
Shuffled confidence	0.735	0.795	0.104
Inverted weighting	0.733	0.797	0.668
$\alpha = 2$ / $\alpha = 4$	0.728 / 0.727	0.793 / 0.788	0.668
Uniform confidence + self-loop	0.742	0.817	<0.001
GCN (reference)	0.740	0.819	0.015

The ablations in Table II locate the cause. Discarding confidence (uniform), shuffling it across nodes or even inverting it to favor the *least* confident neighbors leaves the result unchanged, so the behavior of Conf-GCN comes

from its normalization. Adding a self-loop to the uniform layer recovers GCN-level performance (F1 0.742, AUROC 0.817): the AUROC gap was caused by the node's own features being diluted. At matched recall, a GCN with a tuned threshold is 0.9 points *more* precise than Conf-GCN ($p < 0.001$), so the precision-oriented operating point is not a property of the method. On the real data the confidence weights deviate from uniform aggregation by only 6.5% on average (largest-to-smallest weight in a neighborhood: median 1.45).

B. Graph Construction (RQ3)

Table III shows that changing only the graph reverses the conclusion about GNNs. On the metadata graph every GNN beats the MLP ($p < 2 \times 10^{-5}$ for each), and GraphSAGE becomes the best detector of the study with F1 0.840 and AUROC 0.921, which is 3.2 F1 points above the MLP and 9.9 F1 points above the original description-graph GCN ($p < 10^{-6}$). Precision and recall improve together (0.842 and 0.838 vs. 0.731 and 0.752). GraphSAGE benefits most because its root transformation keeps a node's own features next to the aggregated neighborhood. The density of the graph matters far less: for $k \in \{5, 10, 20\}$ GCN on the description graph varies only between 0.732 and 0.740 F1.

TABLE III
F1 (AUROC) FOR THREE k-NN GRAPHS, $k = 10$, 25 RUNS. EDGE HOMOPHILY: 0.617 / 0.777 / 0.756.

Model	Description	Metadata	Feature
GCN	0.740 (0.819)	0.826 (0.915)	0.820 (0.905)
GraphSAGE	0.803 (0.887)	**0.840 (0.921)**	0.825 (0.906)
GAT	0.734 (0.819)	0.823 (0.912)	0.814 (0.898)
Conf-GCN	0.731 (0.794)	0.824 (0.912)	0.818 (0.901)
MLP (no graph)		0.807 (0.890)	

C. Adaptive Extensions (RQ2)

Applying confidence on top of GCN's own normalization removes the weakness of the original layer: on the description graph AC-GCN reaches F1 0.737 and AUROC 0.819, the same as GCN and 2.5 AUROC points above Conf-GCN. On the feature graph AC-GCN (0.818), the agreement-weighted GCN (0.818) and AC-SAGE (0.822) stay within 0.4 F1 points of GCN (0.820) and GraphSAGE (0.825), and none is significantly better (Holm-corrected $p \geq 0.16$). On the metadata graph AC-SAGE is slightly worse than GraphSAGE (0.835 vs. 0.840). The learned exponents explain why: starting from one, they converge to about 0.6–0.7 for AC-GCN and 0.76–0.78 for AC-SAGE, that is, given the freedom to amplify confidence, the models reduce it. Confidence is only moderately predictive of whether a neighbor is correct (AUROC 0.66 on the description graph), so amplifying it would also amplify errors.

D. Few Labeled Accounts (RQ4)

Labels are expensive in moderation. Fig. 1 shows that the graph matters most when labels are few. GraphSAGE with confidence-based self-training is the best model with 100, 250 and 500 labels (F1 0.764, 0.781 and 0.795 vs. 0.744, 0.761 and 0.773 for the MLP; $p = 0.002$ at 250 and 500 labels, $p = 0.06$ at 100). Self-training adds 0.6 F1 points to GraphSAGE at 250 labels ($p = 0.014$) and about 1 point to GCN. Here confidence is useful, but as a filter for pseudo-labels rather than as a weight inside message passing.

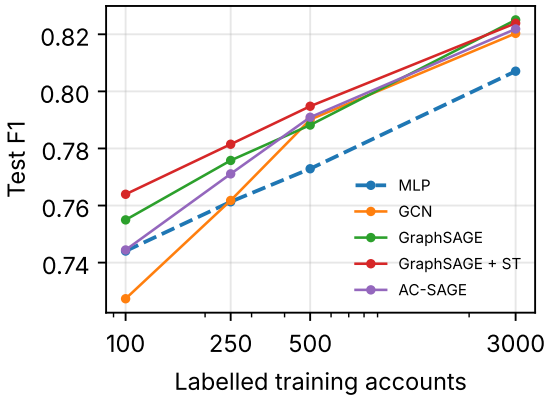


Fig. 1. Test F1 versus the number of labeled training accounts (feature graph; 10 splits for 100–500 labels, 25 runs for 3000).

VI. Discussion

The study yields a simple recipe for bot detection on tabular account data: build the similarity graph from account metadata and use an aggregator that keeps a node's own features; with few labels, add confidence-based self-training. Confidence reweighting inside message passing is cheap and, in its adaptive form, as accurate as its baselines, but in the binary case its weights stay close to uniform, whereas graph construction changes which accounts are neighbors at all. Limitations: the graphs are similarity graphs built from the same features used for classification, labels partly come from Botometer (a metadata-based detector), and we used a 5000-account English-only subsample; real follow or mention graphs may behave differently. Runs share data across folds, so p-values are indicative.

VII. Conclusion

We evaluated confidence-aware message passing for social bot detection with repeated cross-validation, paired tests and ablations. The earlier single-split precision gain of Conf-GCN was sampling noise, and neither learnable exponents nor agreement weights improve on GCN or GraphSAGE. The decisive factor is the graph: a metadata k-NN graph turns every GNN into a better detector than

a feature-only model, and GraphSAGE on it reaches F1 0.840 and AUROC 0.921, 9.9 F1 points above the original pipeline; with only a few hundred labels, confidence-based self-training on this graph gives the best results. Future work will evaluate the recipe on graphs with explicit follow and mention edges such as TwiBot-20 [11] and learn the graph jointly with the classifier.

ACKNOWLEDGMENT

The authors thank the School of Software Engineering at Astana IT University for its support.

REFERENCES

[1] S. Cresci, "A decade of social bot detection," *Communications of the ACM*, vol. 63, no. 10, pp. 72–83, 2020. Available: https://doi.org/10.1145/3409116

[2] S. Ali Alhosseini, R. Bin Tareaf, P. Najafi, and C. Meinel, "Detect me if you can: Spam bot detection using inductive representation learning," in *Companion Proc. 2019 World Wide Web Conference*, 2019, pp. 148–153. Available: https://doi.org/10.1145/3308560.3316504

[3] S. Feng, H. Wan, N. Wang, and M. Luo, "BotRGCN: Twitter bot detection with relational graph convolutional networks," in *Proc. 2021 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining, ASONAM*, 2021, pp. 236–239.

[4] S. Feng, Z. Tan, R. Li, and M. Luo, "Heterogeneity-aware twitter bot detection with relational graph transformers," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 36, no. 4, 2022, pp. 3977–3985.

[5] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. 5th International Conference on Learning Representations, ICLR 2017*, 2017. Available: https://arxiv.org/abs/1609.02907

[6] K. Kassymkhan, "Confidence-aware graph convolutional networks for social bot detection: A message-passing reweighting approach," in *Proc. 2026 IEEE 3rd International Student Conference on Digital Generation*, Astana, Kazakhstan, 2026.

[7] C. A. Davis, O. Varol, E. Ferrara, A. Flammini, and F. Menczer, "BotOrNot: A system to evaluate social bots," in *Proc. 25th International Conference Companion on World Wide Web*, 2016, pp. 273–274. Available: https://doi.org/10.1145/2872518.2889302

[8] K.-C. Yang, O. Varol, P.-M. Hui, and F. Menczer, "Scalable and generalizable social bot detection through data selection," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 34, no. 1, 2020, pp. 1096–1103. Available: https://doi.org/10.1609/aaai.v34i01.5460

[9] D. Lee, S. Lee, and H. Yu, "Discriminative feature selection for multi-class Twitter bot detection," *IEEE Access*, vol. 9, pp. 32 752–32 762, 2021.

[10] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems 30, NeurIPS 2017*, 2017, pp. 1024–1034. Available: https://arxiv.org/abs/1706.02216

[11] S. Feng, H. Wan, N. Wang, J. Li, and M. Luo, "TwiBot-20: A comprehensive twitter bot detection benchmark," in *Proc. 30th ACM International Conference on Information and Knowledge Management, CIKM*, 2021, pp. 4485–4494. Available: https://doi.org/10.1145/3459637.3482019

[12] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. 6th International Conference on Learning Representations, ICLR 2018*, 2018. Available: https://arxiv.org/abs/1710.10903

[13] J. Zhu, Y. Yan, L. Zhao, M. Heimann, L. Akoglu, and D. Koutra, "Beyond homophily in graph neural networks: Current limitations and effective designs," in *Advances in Neural Information Processing Systems 33, NeurIPS 2020*, 2020, pp. 7793–7804. Available: https://arxiv.org/abs/2006.11468

[14] S. Vashishth, P. Yadav, M. Bhandari, and P. Talukdar, "Confidence-based graph convolutional networks for semi-supervised learning," in *Proc. 22nd International Conference on Artificial Intelligence and Statistics, AISTATS*, 2019.

[15] D.-H. Lee, "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks," in *ICML Workshop on Challenges in Representation Learning*, 2013.

[16] Q. Li, Z. Han, and X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018, pp. 3538–3545. Available: https://arxiv.org/abs/1801.07606

[17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th International Conference on Machine Learning, ICML*, 2017, pp. 1321–1330. Available: https://arxiv.org/abs/1706.04599

[18] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conference on Empirical Methods in Natural Language Processing, EMNLP-IJCNLP*, 2019, pp. 3982–3992. Available: https://doi.org/10.18653/v1/D19-1410

[19] X. Zhu, Z. Ghahramani, and J. Lafferty, "Semi-supervised learning using Gaussian fields and harmonic functions," in *Proc. 20th International Conference on Machine Learning, ICML*, 2003, pp. 912–919.

[20] D. M. Gutiérrez, "Twitter human bots dataset," Kaggle, 2021. Available: https://www.kaggle.com/datasets/davidmartngutirrez/twitter-bots-accounts

[21] Bot-Ani project, "Processed Twitter human/bot dataset (English accounts)," GitHub repository, 2024. Available: https://github.com/Max-SF1/Bot-Ani-RWKV-twitter-bot-detection

[22] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. Available: https://arxiv.org/abs/1903.02428

[23] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006. Available: https://jmlr.org/papers/v7/demsar06a.html

[24] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.